

**UNIVERSIDAD INTERNACIONAL DEL
ECUADOR**

**LÓGICA DE PROGRAMACIÓN 1-CIB-
1B**

**PROYECTO FINAL LÓGICA DE
PROGRAMACIÓN**

**TAPIA MONTAÑO STEFANO
JEANPIERRE**

SALAZAR TAPIA MONICA PATRICIA

Contenido

INTRODUCCIÓN.....	3
DESCRIPCIÓN DEL PROBLEMA	3
¿Qué le falta al juego? (Lo que quiero mejorar)	3
CRONOGRAMAS	5
CRONOGRAMA DEL MÓDULO DE LÓGICA DE PROGRAMACIÓN	5
CRONOGRAMA DE ACTIVIDADES DEL MÓDULO DE LÓGICA DE PROGRAMACIÓN	7
DETALLE DE ACTIVIDADES POR SEMANA	9
EXPLICACIÓN DE SISTEMA.....	10
1. El escenario y los actores.....	10
2. El movimiento (El corazón del juego)	10
3. La lógica de las colisiones (La parte más importante)	10
4. La interacción	11
REFLEXIÓN	11
Bibliografía.....	11

INTRODUCCIÓN

Este documento es un resumen de todo lo que he aprendido y aplicado en esta asignatura durante estas siete semanas. El objetivo principal ha sido pasar de la teoría sobre cómo resolver problemas, a aplicar esa lógica directamente en el computador.

Como proyecto principal, decidí recrear el clásico Atari Pong usando Python y la librería turtle. Más que un simple juego, lo veo como la prueba de fuego de todo lo que estudié: aquí es donde conceptos que suenan abstractos —como los bucles infinitos, el manejo de coordenadas y las condiciones lógicas— cobran sentido y se convierten en algo funcional.

A través del desarrollo de este código, he tenido la oportunidad de enfrentar desafíos técnicos reales, tales como la depuración de la lógica de rebote y la sincronización de eventos de entrada (teclado). Este trabajo refleja la evolución de mis capacidades para transformar requisitos técnicos en soluciones funcionales, demostrando que la programación es, en esencia, una disciplina basada en la resolución estructurada de problemas.

A continuación, presento la descripción del problema, el cronograma detallado de las actividades académicas cursadas, la explicación técnica de mi proyecto, una reflexión crítica sobre el papel de la tecnología en nuestra sociedad actual.

DESCRIPCIÓN DEL PROBLEMA

Siendo sincero, lo que más me quebró la cabeza fue **hacer que la pelota rebotara bien al tocar las paletas**. Al principio, pensaba que era solo decir 'si toca, rebota', pero me di cuenta de que el computador no tiene ni idea de qué es una colisión física.

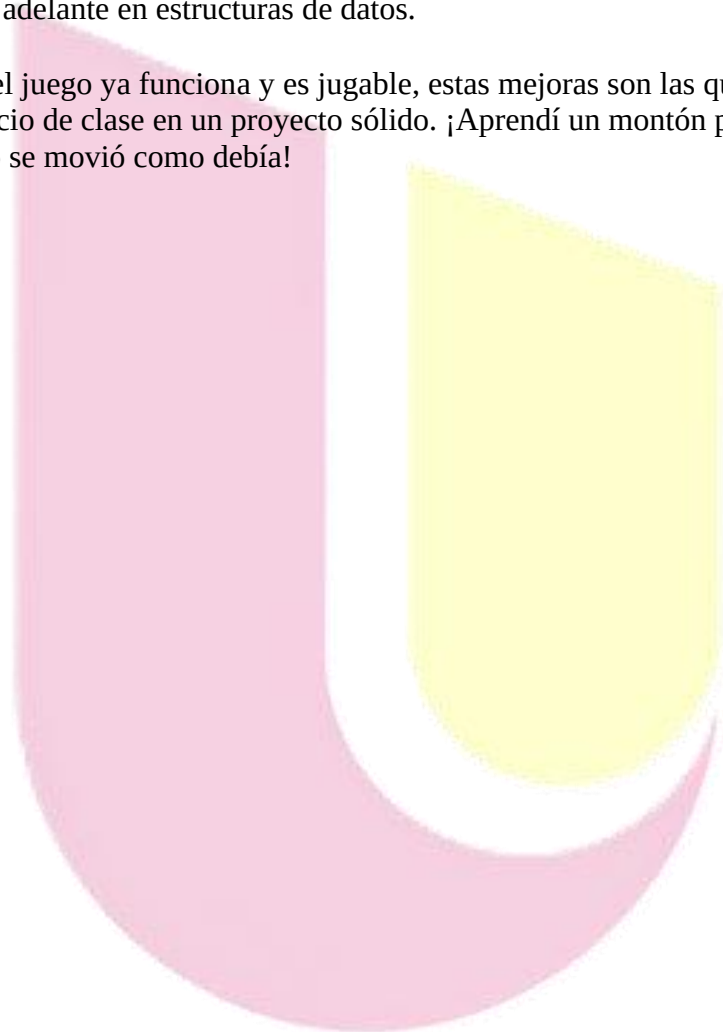
Tuve que ponerme a pensar en coordenadas exactas y usar mucha lógica booleana (los famosos `and` y operadores de comparación). Lo más difícil fue encontrar el punto justo: si definía mal el rango de altura (`ycor`) de la paleta, la pelota simplemente la atravesaba como si fuera un fantasma, o peor aún, se quedaba 'pegada' dentro del objeto porque el programa detectaba la colisión demasiadas veces seguidas. Tuve que ajustar los números una y otra vez hasta que el rebote se sintiera natural. Para mí, ese fue el momento clave donde entendí que programar no es solo escribir código, sino enseñarle al computador a comportarse como yo quiero."

¿Qué le falta al juego? (Lo que quiero mejorar)

Si soy autocrítico, sé que mi código es una versión funcional pero básica. Si quisiera dejarlo de nivel profesional, estas son las cosas que le añadiría:

- **Dificultad progresiva:** Ahora mismo la pelota va siempre a la misma velocidad. Le falta esa lógica de "aumentar la velocidad" cada vez que alguien hace un punto o toca la paleta; eso haría que el juego se pusiera más intenso conforme avanza.
- **Limpieza (Modularidad):** Mi `while True` (el bucle principal) se ve un poco cargado. Me falta organizar mejor el código usando más funciones, para que el bucle solo llame a cosas como `verificar_colision()` o `actualizar_pelota()` y no esté todo mezclado.
- **Un 'Game Over' real:** Actualmente el juego no termina nunca. Estaría genial añadir una estructura de decisión (`if`) que detecte cuando alguien llega a 5 o 10 puntos, detenga el juego y muestre un mensaje de ganador.
- **Un toque extra:** Podría implementar el uso de una lista o un diccionario para gestionar mejor los nombres de los jugadores o incluso guardar la puntuación histórica, algo que veríamos más adelante en estructuras de datos.

En resumen, aunque el juego ya funciona y es jugable, estas mejoras son las que realmente convertirían un ejercicio de clase en un proyecto sólido. ¡Aprendí un montón probando y fallando hasta que por fin todo se movió como debía!



CRONOGRAMAS

CRONOGRAMA DEL MÓDULO DE LÓGICA DE PROGRAMACIÓN

TEMA	ACTIVIDAD	SEMANA 1	SEMANA 2	SEMANA 3	SEMANA 4	SEMANA 5	SEMANA 6	SEMANA 7
Introducción a la resolución de problemas	¿Qué son los problemas?							
	Tipos de problemas							
	Fases para resolver problemas							
	Resolución de problemas mediante computadoras							
Entorno de programación	Introducción al entorno de desarrollo Parte 1							
	Introducción al entorno de desarrollo Parte 2							
	¿Qué es la depuración?							
Manejo de datos	Almacenamiento de datos, variables							
	Almacenamiento de datos, variables, tipos de datos, operadores, prioridad de las operaciones.							
Algoritmos y diagramas de flujo	Herramientas de Solución de Problemas Mediante Programación							
	Algoritmos							
	Diagramas de Flujo							
Lógica de Programación - Condicionales	Introducción a las Estructuras de Decisión							
	Operadores Racionales							
	Operadores Lógicos							

[illegible]

CRONOGRAMA DE ACTIVIDADES DEL MÓDULO DE LÓGICA DE PROGRAMACIÓN

TEMA	CONTENIDO/ACTIVIDAD	SEMANA 1	SEMANA 2	SEMANA 3	SEMANA 4	SEMANA 5	SEMANA 6	SEMANA 7
Introducción a la resolución de problemas	Definición de problemas, sus tipos, pasos fundamentales para resolverlos y el enfoque específico mediante el uso de computadores.							
	Entrega de Tarea Autónoma 1							
Entorno de programación	Introducción al IDE (parte 1 y 2) y los conceptos fundamentales de la depuración (debugging) para corregir errores.							
	Realización de Examen Práctico 2							
Manejo de datos	Almacenamiento, declaración de variables, tipos de datos, operadores aritméticos y la jerarquía de prioridad en las operaciones.							
	Entrega de Tarea Autónoma 2 y Examen Práctico 3.							
Algoritmos y diagramas de flujo	Uso de herramientas de programación, diseño de algoritmos lógicos y construcción de diagramas de flujo.							
	Realización de Examen Práctico 4							
	Estructuras de decisión, uso de operadores relacionales y							

[illegible]

DETALLE DE ACTIVIDADES POR SEMANA

Semana	Temas Desarrollados (Contenido)	Actividades Realizadas
1	Resolución de Problemas: Definición de problemas, sus tipos, pasos fundamentales para resolverlos y el enfoque específico mediante el uso de computadores.	Entrega de Tarea Autónoma 1 (31 de mayo).
2	Entorno de Programación: Introducción al IDE (parte 1 y 2) y los conceptos fundamentales de la depuración (debugging) para corregir errores.	Realización de Examen Práctico 2 (24 de mayo).
3	Manejo de Datos: Almacenamiento, declaración de variables, tipos de datos, operadores aritméticos y la jerarquía de prioridad en las operaciones.	Entrega de Tarea Autónoma 2 (21 de junio) y Examen Práctico 3.
4	Algoritmos y Flujos: Uso de herramientas de programación, diseño de algoritmos lógicos y construcción de diagramas de flujo.	Realización de Examen Práctico 4 (7 de junio).
5	Lógica (Condicionales): Estructuras de decisión, uso de operadores relacionales y lógicos, y la implementación de la estructura condicional "IF".	Realización de Examen Práctico 5.
6	Bucles: Introducción a la iteración, uso de bucles "While" y "For", además del manejo de sentencias que alteran el flujo de los bucles.	Realización de Examen Práctico 6.
7	Estructuras y Funciones: Manejo de Tuplas, Listas, Diccionarios y la creación/ejecución de Funciones con parámetros.	Realización de Examen Práctico 7 (28 de junio) y Tarea de contacto con el docente.

EXPLICACIÓN DE SISTEMA

1. El escenario y los actores

Primero, preparamos la pantalla con turtle. Definimos el lienzo, los colores y colocamos a los tres actores principales: la paleta A (izquierda), la paleta B (derecha) y la pelota en el centro. También creamos un "lápiz" invisible que es el que se encarga de ir escribiendo el marcador en la parte superior cada vez que alguien anota un punto.

2. El movimiento (El corazón del juego)

El código usa un `while True`, que es básicamente un motor que corre miles de veces por segundo. En cada vuelta de este motor, ocurren tres cosas:

Actualizamos posiciones: La pelota suma un poquito a sus coordenadas X e Y (`pelota.dx` y `pelota.dy`), lo que hace que se mueva por la pantalla.

Rebote en paredes: Tenemos unos `if` que vigilan las coordenadas de la pelota. Si toca el borde superior o inferior (más de 290 o menos de -290), multiplicamos la velocidad vertical por -1. Matemáticamente, esto hace que la pelota cambie de dirección al instante.

Gestión de puntos: Si la pelota se sale por la izquierda o la derecha (más de 390 o menos de -390), el programa entiende que alguien falló. Reseteamos la pelota al centro, sumamos un punto al marcador y usamos `lapiz.clear()` para borrar el puntaje viejo y escribir el nuevo.

3. La lógica de las colisiones (La parte más importante)

Esta es la parte donde le enseñamos a la pelota a "chocar" contra las paletas. Como el computador no sabe qué es una paleta, yo le digo:

"Si la pelota está en una posición X entre 340 y 350 (justo donde está la paleta) Y ADEMÁS su altura está dentro del rango de la paleta (es decir, a menos de 50 píxeles del centro de la paleta), entonces invierte la dirección en X".

Es como si estuviéramos estableciendo un área de choque matemática. Sin esto, la pelota simplemente pasaría de largo.

4. La interacción

Finalmente, tenemos las funciones de control (paleta_a_arriba, paleta_b_abajo, etc.). Estas están "escuchando" el teclado. Cuando presionas una tecla (W, S, o las flechas), el programa simplemente le suma o resta 20 píxeles a la posición Y de la paleta, siempre y cuando no se salgan de los bordes de la pantalla.

En resumen: Es una secuencia constante de revisar posición -> verificar colisión -> actualizar pantalla. Es un ciclo rápido que crea la ilusión de movimiento fluido que ves en pantalla.

REFLEXIÓN

La tecnología ha dejado de ser un simple complemento para convertirse en el eje sobre el cual gira nuestra sociedad. Su impacto es innegable: ha eliminado distancias, democratizado el acceso a la información y optimizado tareas que antes nos tomaban días. Sin embargo, este progreso no está exento de retos.

Como estudiante de programación, entiendo que la tecnología es un arma de doble filo. Por un lado, es un motor de innovación capaz de resolver problemas complejos; por otro, su uso desmedido o poco ético puede ampliar brechas sociales y generar dependencia. El verdadero desafío no está en la herramienta en sí, sino en nuestra capacidad para gobernarla.

En conclusión, la tecnología debe ser vista como un medio para potenciar nuestras capacidades humanas, no para reemplazarlas. El impacto positivo dependerá siempre de la ética y el pensamiento crítico con los que decidamos integrarla en nuestra vida diaria. En última instancia, la tecnología nos da el "cómo", pero nosotros seguimos siendo los únicos responsables de definir el "para qué".

Bibliografía

Tecnología en la educación - 2021/2 GEM Report. (2023, julio 23). 2021/2 GEM Report.

<https://gem-report-2023.unesco.org/es/tecnologia-en-la-educacion/>